Assignment - 1

Analysis of performance of 3 sorting algorithms:

- Insertion sort
- Merge Sort
- Quick Sort

Date Of Performance: 22/06/26

Date of Submission: 23/06/26

Name : Sakif Ahmed Rafi

Student ID : 00724105101024

Group : A1

Objective

Given a sorted array of size 10^5. (sorted_random.txt). Compare the performance of three sorting algorithms -

i) Insertion Sort ii) Merge Sort iii) Quick Sort

And report the results found from analysis.

Tasks

1. Array Input
   - Read the file sample.txt
   - Use all the samples as input
   - Store the numbers in an array

2. Algorithm Implementation
   - Implement Insertion Sort, Merge Sort, Quick Sort

3. Performance Measurement

   For each run, measure:

   Execution time (s)
   Estimated energy consumption (J)
   Energy (J) = CPU's approximate average power x Execution Time (s)
   Peak memory usage (KB)
   - Estimated CO_2 emissions (Bangladesh)
   Energy (kWh) = Energy (J) / 3,600,000
   CO_2 (kg) = Energy (kWh) x 0.62

Deliverables

- Source code implementing insertion sort, merge sort and quick sort.
- Filled results table and discussion.

Algorithm	Array Size	Execution Time (s)	Energy (J)	Peak Memory (KB)	CO_2 Emissions (kg, BD)
Insertion Sort	10^5	0.049628	3.23	4012	0.00000056
Merge Sort	10^5	0.067938	4.42	4004	0.00000075
Quick Sort	10^5	3.129432	182.91	6987	0.00004119

Discussion / Conclusion

- **Insertion Sort**: Best overall for array size 10^5. Achieved lowest execution time, energy consumption, CO_2 emissions, and memory usage. Most energy-efficient and environmentally friendly.
- **Merge Sort**: Slightly higher time and energy than Insertion Sort, same memory. Moderate CO_2 emissions. Good balance between performance and resources.
- **Quick Sort**: Highest execution time, energy, CO_2 emissions, and memory usage. Performed poorly due to pivot strategy or input data.
- **Overall**: Insertion Sort is the most carbon-efficient. Merge Sort offers reasonable trade-off. Quick Sort needs optimization (better pivot selection) to reduce environmental impact.

Code for Performance Measurement

```
#include
<bits/stdc++.h>
using namespace
std;

int main() {

    int n;
    cin >> n;
    vector<int>
vec(n);
    for(int
i=0; i<n; i++)
        cin >>
vec[i];

    for(int
i=1; i<n; i++){
        int
temp = vec[i];
        int ind
= i;
        for(int
j=i-1; j>=0; j-
-){
            if(
vec[j]>temp){

 vec[j+1] =
vec[j];

 ind = j;
            }
```

```
        else break;
        }
        vec[ind] = temp;
    }

    for(auto x : vec) cout << x << " ";

    return 0;
}


#include <bits/stdc++.h>
using namespace std;

vector<int> vec;

void merge(int l, int mid, int h){
    vector<int> ans;
    int i = l, j = mid+1;
    while(i<=mid && j<=h){
        if(vec[i]<=vec[j]){
            ans.push_back(vec[i]);
            i++;
        }
        else{
            ans.push_back(vec[j]);
            j++;
        }
    }
    while(i<=mid){
        ans.push_back(vec[i]);
        i++;
    }
    while(j<=h){
        ans.push_back(vec[j]);
        j++;
    }
    for(int i=l,j=0; i<=h; i++,j++){
        vec[i] = ans[j];
    }
}

void merge_sort(int l, int h){
    if(l<h){
        int mid = (l+h)/2;
        merge_sort(l,mid);
        merge_sort(mid+1,h);
        merge(l,mid,h);
    }
}

int main() {
```

```
    int n,x;
    cin >> n;
    for(int i=0; i<n; i++){
        cin >> x;
        vec.push_back(x);
    }

    merge_sort(0,n-1);
    for(auto x : vec) cout << x << " ";

    return 0;
}


#include <bits/stdc++.h>
using namespace std;

vector<int> vec;

void quick_sort(int l, int h){
    if(l<h){
        int pivot = vec[l];
        int i = h+1;
        for(int j=h; j>l; j--){
            if(vec[j]>=pivot){
                i--;
                swap(vec[i],vec[j]);
            }
        }
        swap(vec[i-1],vec[l]);
        int pi = i-1;
        quick_sort(l,pi-1);
        quick_sort(pi+1,h);
    }
}

int main() {

    int n,x;
    cin >> n;
    for(int i=0; i<n; i++){
        cin >> x;
        vec.push_back(x);
    }

    quick_sort(0, n-1);


    return 0;
}
```